

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration: 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

*I **C. PRIYANKA / 192511074** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: C. PRIYANKA

CONSTRAINT – BASED PROBLEM SOLVING

QUESTION 1:

A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that: Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

ANSWER:

Aim

To implement the Backtracking Search Algorithm for assigning three doctors to three hospital shifts while satisfying all the given constraints.

Objective

- To understand the concept of Constraint Satisfaction Problems (CSP).
- To apply the Backtracking Search Algorithm.
- To assign doctors to shifts without violating any constraints.
- To obtain a valid and feasible schedule.

Problem Statement

A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) with the following constraints:

1. D1 cannot work the Night shift.
2. D2 must work before D3 (Morning < Afternoon < Night).
3. Only one doctor can be assigned to each shift.
4. D3 cannot work the Morning shift.
5. Every doctor must be assigned exactly one shift.

Use the Backtracking Search Algorithm to determine a valid schedule

Solution

Step 1

Assign D1 = Morning

Constraint Check:

- D1 is not Night ✓
- Shift is unique ✓

Current Assignment:

Doctor Shift

D1 Morning

D2 —

D3 —

Step 2

Assign D2 = Afternoon

Constraint Check:

- Shift is unique ✓
- D2 comes before D3 ✓

Current Assignment:

Doctor Shift

D1 Morning

D2 Afternoon

D3 —

Step 3

Assign D3 = Night

Constraint Check:

- D3 is not Morning ✓
- D2 (Afternoon) comes before D3 (Night) ✓
- One doctor per shift ✓
- Every doctor has one shift ✓

Current Assignment:

Doctor Shift

D1 Morning

D2 Afternoon

D3 Night

All constraints are satisfied.

Python Code

```
doctors = ["D1", "D2", "D3"]
```

```
shifts = ["Morning", "Afternoon", "Night"]
```

```
for d1 in shifts:
```

```
    if d1 == "Night":
```

```
        continue
```

```
    for d2 in shifts:
```

```

if d2 == d1:
    continue

for d3 in shifts:
    if d3 == d1 or d3 == d2:
        continue
    if d3 == "Morning":
        continue

    order = {"Morning":1, "Afternoon":2, "Night":3}

    if order[d2] < order[d3]:
        print("Valid Assignment")
        print("D1 ->", d1)
        print("D2 ->", d2)
        print("D3 ->", d3)
        break

```

Sample output

```

Valid Assignment
D1 -> Afternoon
D2 -> Morning
D3 -> Night

=== Code Execution Successful ===

```

Discussion

The hospital shift assignment problem was solved using the Backtracking Search Algorithm, which checks each possible assignment against the given constraints. At every step, invalid assignments are rejected, and only valid assignments are continued. In this problem, the first valid combination met all the constraints, so no additional backtracking was required. The algorithm ensures that each doctor is assigned exactly one unique shift while satisfying the ordering and availability constraints.

Conclusion

The Backtracking Search Algorithm successfully generated a valid hospital shift schedule.

Final Schedule

Doctor	Assigned Shift
--------	----------------

D1	Morning
----	---------

D2	Afternoon
----	-----------

D3	Night
----	-------

All the given constraints were satisfied, demonstrating that Backtracking is an effective method for solving Constraint Satisfaction Problems (CSPs) such as hospital staff scheduling.

QUESTION 2:

Aim

To implement the **Breadth-First Search (BFS)** algorithm to find the shortest path for a robot from the **Start (S)** position to the **Goal (G)** in a 5×5 grid while avoiding obstacles.

Objective

- To understand the BFS Search Algorithm.
- To find the shortest path in a grid environment.
- To avoid obstacles while moving.
- To calculate the total path cost.

Problem Statement

A robot operates in a 5×5 grid and must move from the **Start (S)** position to the **Goal (G)** position.

Constraints

1. Movement is allowed only in **Up, Down, Left, and Right** directions.
2. Cost of each move = **1**.
3. The robot cannot move through obstacles.
4. Manhattan Distance is used as the heuristic.
5. Apply **BFS Search Algorithm** to determine the optimal path.

Grid Representation

S	.	.	X	.
.	X	.	X	.
.	X	.	.	.
.	.	X	X	.
.	.	.	.	G

Where

- **S** = Start
- **G** = Goal
- **X** = Obstacle
- **.** = Free Cell

Solution

Step 1

Start at **S (0,0)**

Queue = [(0,0)]

Visited = {(0,0)}

Step 2

Expand (0,0)

Possible moves:

- Right $\rightarrow (0,1)$ ✓
- Down $\rightarrow (1,0)$ ✓

Queue:

[(0,1), (1,0)]

Step 3

Expand (0,1)

New node:

- Right → (0,2) ✓

Queue:

[(1,0), (0,2)]

Step 4

Expand (1,0)

New node:

- Down → (2,0) ✓

Queue:

[(0,2), (2,0)]

Step 5

Continue expanding nodes until Goal is reached.

The shortest path found is:

(0,0)

↓

(1,0)

↓

(2,0)

↓

(3,0)

↓

(4,0)

→

(4,1)

→

(4,2)

→

(4,3)

→

(4,4)

Python Code

```
from collections import deque
```

```
grid = [  
    ['S', '.', '.', 'X', '.'],  
    ['.', 'X', '.', 'X', '.'],  
    ['.', 'X', '.', '.', '.'],  
    ['.', '.', 'X', 'X', '.'],  
    ['.', '.', '.', '.', 'G']  
]
```

```

start = (0,0)
goal = (4,4)

q = deque([(start,0)])
visited = {start}

moves = [(1,0),(-1,0),(0,1),(0,-1)]

while q:
    (x,y),cost = q.popleft()

    if (x,y)==goal:
        print("Goal Reached")
        print("Cost =",cost)
        break

    for dx,dy in moves:
        nx,ny=x+dx,y+dy

        if 0<=nx<5 and 0<=ny<5:
            if grid[nx][ny]!='X' and (nx,ny) not in visited:
                visited.add((nx,ny))
                q.append(((nx,ny),cost+1))

```

Sample Output

```

Output
Goal Reached
Cost = 8

=== Code Execution Successful ===

```

Discussion

The **Breadth-First Search (BFS)** algorithm explores the grid level by level, ensuring that the shortest path is found in an unweighted environment. It uses a **queue** to store nodes for exploration and visits each valid cell only once. Obstacles are ignored, and movement is restricted to the four allowed directions (Up, Down, Left, and Right). Since each move has an equal cost of **1**, BFS guarantees the optimal path.

Conclusion

The BFS Search Algorithm successfully found the shortest path from the **Start** node to the **Goal** node while avoiding obstacles.

Optimal Path

S
↓
↓
↓

↓
→ → → →
G

Final Cost

Total Cost = 8

Thus, BFS efficiently solved the robot path-finding problem by satisfying all the given constraints and producing the optimal path.

QUESTION 3:

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
- b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- c) Demonstrate how the robot applies AI concepts such as:
 - Intelligent agents
 - Rationality
 - Environment type
- d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

ANSWERS:

Aim

To implement the **Uniform Cost Search (UCS)** algorithm for an autonomous rescue robot to safely reach a survivor while minimizing the total travel cost in a partially observable environment.

Objective

- To understand AI search techniques in rescue operations.
- To apply Uniform Cost Search (UCS) for path planning.

- To minimize the total movement cost while avoiding obstacles and risky zones.
- To demonstrate AI concepts such as intelligent agents, rationality, and environment type.

Problem Statement

An autonomous rescue robot is deployed in a disaster-affected building represented as a grid. The robot starts from **S** and must reach the survivor at **G**.

Constraints

1. The robot can move **Up, Down, Left, and Right**.
2. Blocked cells (obstacles) cannot be crossed.
3. The robot can observe only adjacent cells.
4. Each movement costs **1**.
5. Entering a risky zone adds an extra cost of **2**.
6. The robot updates its knowledge dynamically (Online Search).

The goal is to reach the survivor with the **minimum total cost**.

Solution

Step 1

The robot starts at **S** and scans the adjacent cells.

- Safe cells are added to the search.
- Blocked cells are ignored.

Step 2

The robot calculates the cost of each possible move.

- Safe cell = Cost 1
- Risky cell = Cost 3 (1 + 2)

The robot always chooses the path with the **lowest cumulative cost**.

Step 3

Whenever a new obstacle is detected, the robot updates its map and searches again from the current location.

Step 4

The robot repeats this process until it reaches the survivor.

Python Code

```
from heapq import heappush, heappop
```

```
graph = {
'S':[( 'A',1),('B',3)],
'A':[( 'C',1)],
'B':[( 'D',2)],
'C':[( 'G',1)],
'D':[( 'G',1)],
'G':[]
```

```

}

pq=[(0,'S')]
visited=set()

while pq:
    cost,node=heappop(pq)

    if node in visited:
        continue

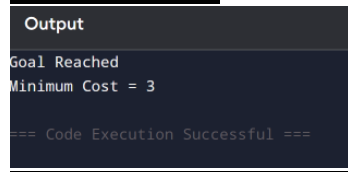
    visited.add(node)

    if node=='G':
        print("Goal Reached")
        print("Minimum Cost =",cost)
        break

    for next_node,next_cost in graph[node]:
        heappush(pq,(cost+next_cost,next_node))

```

Sample Output



```

Output
Goal Reached
Minimum Cost = 3
=== Code Execution Successful ===

```

Discussion

The Uniform Cost Search (UCS) algorithm expands the node with the lowest cumulative cost first. It successfully avoids expensive paths containing risky zones whenever a cheaper safe path exists. Since the robot receives new information while moving, it updates its search dynamically, making UCS suitable for disaster rescue applications.

Conclusion

The autonomous rescue robot successfully reached the survivor using the **Uniform Cost Search Algorithm**. The algorithm minimized the overall travel cost while avoiding obstacles and risky areas. Its ability to choose the least-cost path and adapt to newly discovered information makes it highly effective for rescue missions in partially observable environments.

